

LineJump

Audit Every MCP Server Before It Reaches Your Model

Static Security Analysis for Model Context Protocol Servers

Technical Documentation

Ritvik Indupuri

Abstract

LineJump is a security analysis platform for Model Context Protocol server definitions. It inspects declared tools, descriptions, schemas, prompts, resources and server-level instructions before those definitions are supplied to a model. The primary scanner is a deterministic TypeScript engine designed for repeatable findings, stable rule identifiers and policy-controlled CI gates.

The project combines a React and TanStack Start web application, a headless CLI, local stdio MCP discovery, guarded HTTP retrieval, SQLite persistence, scan comparison, signed report metadata and SARIF, JSON and PDF exports. Optional Ollama-backed analysis adds semantic review and manifest-drift recommendations without replacing the deterministic result.

Document scope

- System boundaries, user interfaces and principal data flows.
- Manifest acquisition through HTTP, local files, stdin, stdio and MCP configuration files.
- Detection, normalization, scoring, policy and cross-tool analysis.
- Optional local-model review, manifest approvals and drift governance.
- Persistence, exports, CI usage, deployment and source cross-reference.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Scan and agent lifecycle	6
4 Detection engine	7
5 Acquisition and transport controls	8
6 Policy, scoring and CI	9
7 Local model analysis and drift governance	9
8 Data architecture and attestation	10
9 Application surfaces and outputs	10
10 Deployment and operation	11
11 Implementation cross-reference	12
12 Next steps	12

1 System overview

LineJump performs pre-flight analysis. It reasons over metadata that an MCP client would expose to a model; it does not invoke the discovered tools as part of the static scan. This makes the core result reproducible and allows the same policy to be used in a browser review or a build pipeline.

The deterministic engine is the baseline security decision. Local model features are optional analyses that require Ollama and a selected model; their output must be reviewed as advisory evidence.

1.1 Implemented capabilities

Capability	Implementation
Manifest inspection	Recursively analyzes MCP tools, input schemas, prompts, resources and server-level text.
Rule engine	Detects injection, obfuscation, dangerous capabilities, permissive schemas and compound attack paths.
Acquisition	Accepts pasted JSON, files, stdin, HTTP endpoints, stdio processes and named MCP configuration entries.
Governance	Merges default and organization policy, stores history and supports approval or denial of manifest drift.
Delivery	Presents web findings and creates terminal, SARIF 2.1.0, JSON and PDF outputs.

1.2 Technology profile

Layer	Technology
Web	React 19, TanStack Router, TanStack Start, Tailwind CSS 4, Radix UI
Server	TanStack server functions, Nitro, Node.js or Bun runtime
Analysis	TypeScript deterministic detection engine; optional Ollama models
State	better-sqlite3 with WAL and foreign-key enforcement
CLI and tests	tsx, Node child_process, Vitest

2 System architecture

Figure 1 separates acquisition, application processing, optional local-model analysis and evidence delivery. Agent icons identify model-backed roles only.

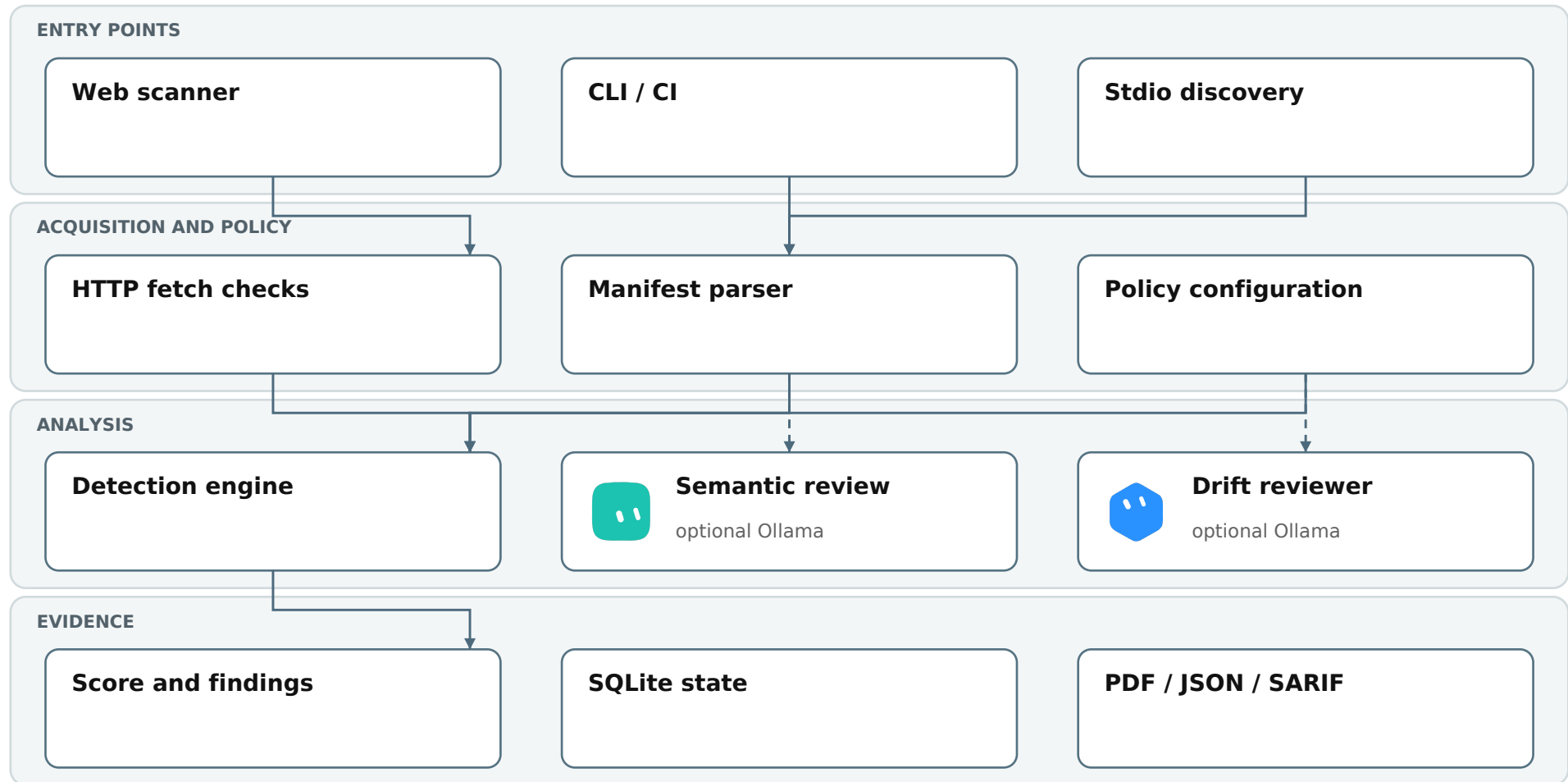


Figure 1: LineJump logical architecture and analysis boundaries.

2.1 Acquisition and canonicalization

LineJump accepts manifests from the browser, CLI, continuous-integration jobs, local stdio servers and named MCP configurations. These entry points converge on the same canonical representation of tools, prompts, resources and server instructions. Remote retrieval is isolated behind guarded HTTP handling, while stdio discovery starts a configured local process only long enough to complete initialization and request its declared tools.

2.2 Deterministic analysis boundary

The detection engine is the authoritative scanner. It evaluates normalized text, nested schemas, declared capabilities and cross-tool relationships under the merged policy. SQLite stores users, scan history, policy state, approvals and audit records. Report services convert the result into the web finding view, CI status and export formats without changing the underlying finding set.

2.3 Optional model services

Local semantic review and manifest-drift recommendations branch from the deterministic path. Both are optional Ollama-backed analyses and are represented with agent icons because they invoke a model. Their recommendations supplement the rule-engine evidence; they do not replace the stable rule identifiers, policy decision or safety score produced by the core scanner.

2.4 Evidence delivery

A completed scan produces a 0-100 safety score, structured findings and attestation metadata. The same result can be rendered in the browser, converted into a threshold-based CI exit status, exported as SARIF, JSON or PDF, and compared with prior scans. This shared result model keeps interactive and pipeline decisions consistent.

3 Scan and agent lifecycle

Figure 2 separates deterministic report generation from optional local-model analysis. Saving, export and CI checks are available output paths; each workflow uses the relevant subset. Connector labels sit inside opaque gaps so every label remains separate from its line.

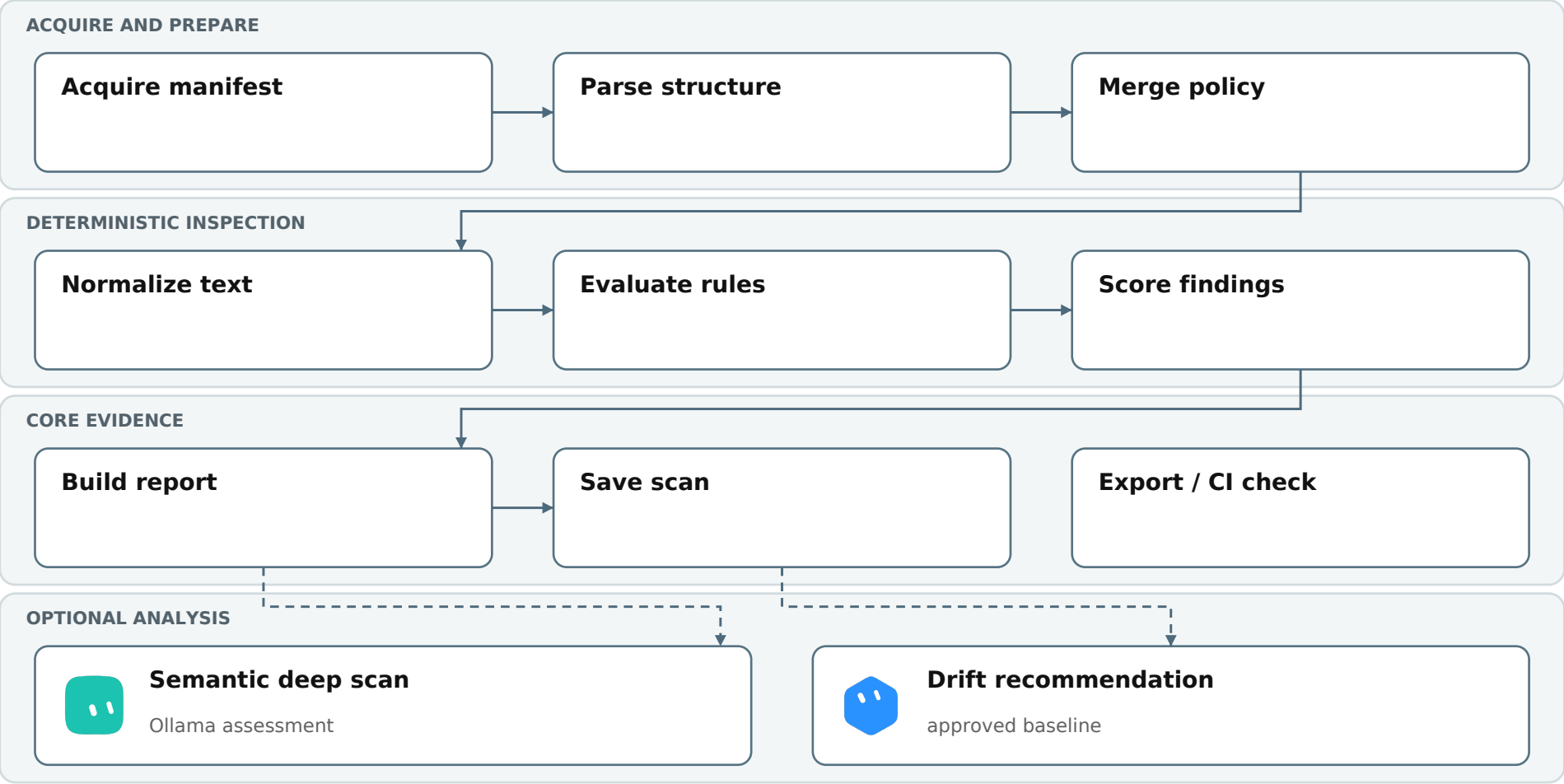


Figure 2: Manifest acquisition, deterministic evaluation, optional local agents and evidence output.

3.1 Core scan sequence

After acquisition, LineJump unwraps supported MCP response shapes and merges the default policy with organization overrides. The scanner builds a context containing the declared tools, prompts and resources, then normalizes text to expose Unicode, encoded and split-field bypass forms. Deterministic rules evaluate content, schema shape and capability combinations before duplicate findings are removed and confidence weights are applied.

Model-chain analysis joins evidence that would be incomplete when fields or tools are examined independently. This enables the scanner to identify compound paths such as one tool reading sensitive data and another transmitting it, or prompt-injection fragments distributed across multiple descriptions.

3.2 Optional agent branches

A semantic deep scan can send the canonical manifest to a configured local Ollama model for additional interpretation. A separate drift reviewer can compare the current declaration with an approved manifest and recommend how the change should be handled. These branches are dashed in the diagram to show that the core scan completes without them.

3.3 Decision evidence

The lifecycle ends with the safety score, structured findings, signed report metadata and persistence or export. Rule identifiers and evidence locations make each result explainable, while scan history supports later comparison. Agent icons appear only on the two local model services; acquisition, normalization, scoring and export remain deterministic application stages.

4 Detection engine

4.1 Analysis surface

The engine converts accepted JSON shapes into a canonical manifest, then inspects tool names, descriptions and nested schema fields alongside prompts, resources and high-level instructions. Text extraction walks schema combinations and other nested values so human-readable instructions are not limited to top-level descriptions.

4.2 Normalization and evasion resistance

- Unicode NFKC normalization and selected homoglyph folding.
- Leetspeak, punctuation and spaced-letter normalization for common bypass forms.
- Explicit checks for zero-width and control characters, ANSI escapes, HTML comments or entities, Markdown JavaScript links, full-width text, Base64 and escaped sequences.
- Split-field analysis that joins text distributed across descriptions and schema fields.

4.3 Finding classes

Class	Representative coverage
Prompt injection	System markers, coercion, priority hijacking, instruction override and multi-phrase combinations.
Capabilities	Shell execution, destructive actions, privilege, secrets, filesystem, network and environment access.
Schema risk	Command, path, URL and query parameters; open additionalProperties; unusual or sensitive tool names.
Compound risk	Cross-tool read-and-send paths and injection fragments split across multiple fields.
External references	Insecure URLs, sensitive resource locations and potentially unsafe path declarations.

4.4 Result model

Each finding contains a stable rule ID, severity, category, title, detail, evidence, location and optional tool name. Duplicate hits for the same context are removed before policy and confidence weighting produce the final 0-100 safety score.

5 Acquisition and transport controls

5.1 Browser and HTTP path

The web scanner accepts pasted JSON or a remote endpoint. The server validates HTTP/S URLs, checks selected local and private address patterns, resolves IPv4 records and makes up to five GET attempts in its redirect loop. The fetch call has an eight-second timeout until response headers arrive; body consumption is outside that timer. The checks inspect the first resolved IPv4 address and do not pin the connection to that address, so they are screening controls rather than a complete SSRF isolation boundary. It first attempts JSON retrieval, then falls back to an MCP tools/list POST and can parse JSON or server-sent event data. The fallback POST uses normal fetch redirect behavior rather than the GET loop's per-hop screening.

The HTTP fetch report records resolution and redirect information. The TLS protocol and issuer fields are hard-coded placeholders, not observations from the connection.

5.2 CLI and stdio path

The CLI reads a file, stdin or a remote URL directly. For a local MCP server it spawns the configured process, performs the initialize and tools/list JSON-RPC sequence over stdio, captures the declared tools and terminates the child process after discovery. Cursor- or Claude-style MCP configuration files can be resolved by server name.

5.3 Trust boundary

Static scanning never calls a discovered MCP tool. Stdio discovery does execute the configured server process so it can request metadata; that path should therefore be used only with a server command the operator already trusts enough to start in the local environment.

6 Policy, scoring and CI

6.1 Policy model

Control	Effect
disabledRules	Removes exact rule IDs from the effective result.
severityOverrides	Changes the severity assigned to selected rules.
blockedCapabilities	Escalates matching capability findings to critical.
customRegexes	Adds organization-defined text patterns and categories.

6.2 CI thresholds

The CLI evaluates the report against maximum critical, high and medium counts plus a minimum score. CI mode prints a Markdown summary and exits with status 1 when any configured threshold is exceeded. Default code values are zero critical, zero high, up to 100 medium and a minimum score of 50.

```
npx tsx cli/scan.ts ./manifest.json --ci \  
+ --max-critical=0 --max-high=0 --min-score=70  
  
npx tsx cli/scan.ts ./manifest.json --sarif results.sarif.json
```

6.3 Determinism

Given the same normalized manifest and effective policy, the core scan produces the same rule matches and score. This property supports repeatable regression review and makes rule IDs suitable for policy-as-code exceptions.

7 Local model analysis and drift governance

7.1 Semantic deep scan

The deep-scan server function sends the manifest and a structured security prompt to a local Ollama endpoint at localhost:11434. Llama Guard 3 is the default model. Returned JSON is normalized into semantic findings, a model score and an executive analysis. Connection or parsing failures return an informational finding and a score of -1 rather than replacing the deterministic report.

7.2 Autonomous manifest reviewer

A second local-model function compares an incoming manifest with the latest approved version and detected structural differences. The response contains a safe or unsafe recommendation, signer metadata and an explanation. The web interface lets a reviewer approve or deny the manifest and stores that status in SQLite.

7.3 Decision authority

The local models provide semantic context and drift advice. Their output is non-deterministic and should be assessed separately from the stable rule report. Approval records provide governance evidence; they do not cause LineJump to execute or proxy a discovered tool.

8 Data architecture and attestation

The SQLite layer opens linejump.db, enables write-ahead logging, applies a five-second busy timeout and enforces foreign keys. The schema supports authentication, scans, policy, CI settings, audit data, catalog entries, pinned tools, proxy observations, quarantined responses and manifest approvals.

8.1 Principal records

Record	Purpose
users / sessions / teams	Local identity, session expiry and team association.
scans	Manifest, score, deterministic findings and optional deep-scan fields.
policies / ci_configs	Organization rules and per-user pipeline thresholds.
audit_logs / proxy_logs	Application actions and observed proxy request metadata.
manifest_approvals	Approved or denied manifest hashes, content, reviewer and key scheme.
pinned_tools / quarantined_responses	Review state for selected tools and held response content.

8.2 Report signing

The attestation module creates a 2048-bit RSA key pair when the server module loads and signs serialized report payloads with SHA-256. The returned signature and public key prove consistency with that running process; the ephemeral key is not a long-term organizational identity and is replaced when the process restarts.

9 Application surfaces and outputs

Surface	Implemented behavior
/app	Manifest input, live fetch, scan results, safety score, findings and ATLAS visualization.
/policy	Policy configuration and persistence.
/history	Stored scans and selection for comparison.
/diff	Side-by-side comparison of two scan records.
CLI	Terminal output, JSON, SARIF and threshold-gated CI mode.
PDF	Human-readable report generated from a scan result.

9.1 SARIF

The SARIF generator produces version 2.1.0 output suitable for code-scanning ingestion. Findings retain rule identity, message, severity mapping and the scanned target URI.

9.2 History and comparison

Web scans can be stored with their manifest and report. The history and diff routes allow two records to be compared, supporting change review across revisions of an MCP server definition.

10 Deployment and operation

10.1 Local development

```
npm install
npm run dev

# production bundle
npm run build
npm run preview
```

The Vite development server is configured by the project, while the container image builds with Bun and serves the production application on port 3000. Docker Compose mounts a persistent data volume for application state.

10.2 Optional model runtime

The core scanner does not require an external AI key. Semantic and drift-review features require a local Ollama service and the relevant model image. The example environment variables for Anthropic are placeholders and are not used by the implemented local-model functions.

10.3 Operational guidance

- Treat manifests as untrusted data and keep the HTTP fetch service isolated from sensitive networks.
- Run stdio discovery only for reviewed commands and constrain the account and filesystem available to the child process.
- Persist the SQLite database on protected storage and limit access to exported findings and manifest content.
- Pin and review policy changes because severity overrides and disabled rules directly affect CI outcomes.

11 Implementation cross-reference

Source reference: LineJump, revision d3dcf487950f. The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

The following map connects the documented behavior to the project source. Generic interface primitives are omitted.

Capability	Primary implementation
Manifest contracts and scan orchestration	src/lib/mcp-scanner.ts
Deterministic rules and scoring	src/lib/detection/engine.ts; src/lib/detection/normalize.ts
Safe HTTP acquisition	src/lib/mcp-fetch.functions.ts
Stdio and MCP configuration discovery	src/lib/mcp-stdio.ts; cli/scan.ts
Policy defaults and persistence	src/lib/default-policy.ts; src/lib/policy.functions.ts; src/lib/db.ts
Local semantic and drift agents	src/lib/deep-scan.functions.ts
Report signing	src/lib/attestation.ts; src/lib/attestation.functions.ts
SARIF, PDF and CI outputs	src/lib/sarif.ts; src/lib/report-pdf.ts; src/lib/ci-check.ts
History, comparison and scanner UI	src/routes/history.tsx; src/routes/diff.tsx; src/routes/app.tsx

12 Next steps

Priority	Improvement	Value
1	Expand regression fixtures	Exercise every stable rule ID, nested schema form and split-field bypass.
2	Harden fetch telemetry	Report certificate facts from the actual connection and add explicit payload-size limits.
3	Persist signing identity	Use a protected long-lived key or KMS-backed signer for durable attestation.
4	Sandbox stdio discovery	Constrain spawned MCP servers with operating-system isolation and resource limits.
5	Separate model scores	Keep local-model findings visibly distinct from deterministic severity and CI policy.